

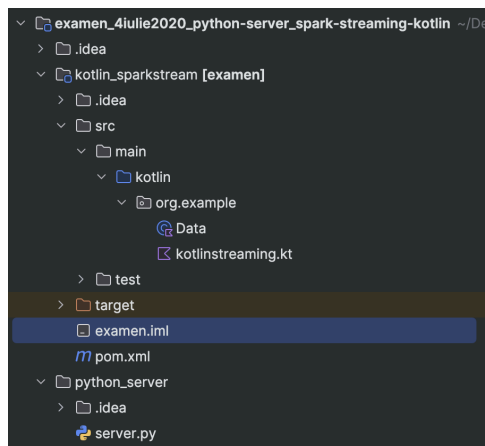
Să se creeze în Python un server TCP care să facă o cerere pentru simbolurile companiilor cu acțiuni (vezi documentația: <https://finnhub.io/docs/api#stock-symbols>) și folosind aceste simboluri precum și API-ul de profil gratuit al unei companii (<https://finnhub.io/docs/api#company-profile2>) să trimită datele prin socket câte un profil o dată la 3 secunde (pentru serializare/deserializare se pot utiliza funcțiile loads și dumps din modulul json din Python, iar în Kotlin se poate utiliza `kotlinx.serialization` <https://github.com/Kotlin/kotlinx.serialization>). De asemenea, se va implementa în Kotlin un flux de date direct (direct stream) utilizând framework-ul Apache Spark, care să preia datele de la serverul TCP și să le prelucreze astfel:

- se vor filtra acele companii care sunt listate pe exchange-ul "NEW YORK STOCK EXCHANGE, INC.";
- se vor filtra acele companii cu un IPO (initial public offer) din anul 2015 până în prezent;
- pentru fiecare RDD în parte, se va afișa numele companiei, numărul de telefon și piața de capital (marketCapitalization).

Există o limitare de 60 de apeluri / minut la fiecare cheie API. Se poate crea un cont pe platforma finnhub.io (<https://finnhub.io/register>), sau se poate utiliza următoarea cheie (token) API: `brmr7v7rh5rcss140lq0`

NOTITE:

1. Pornesti server-ul si dupa rulezi kotlinstreaming.kt
2. Posibil sa ii dea eroare ca nu are pachetul requests(stanga jos la pachete il cauti si il instalezi)
3. Trebuie sa ai un JDK sub 17



Data.kt

```
package org.example

import kotlinx.serialization.Serializable
```

```

@Serializable
data class Data(
    var name: String,
    var phone: String,
    var ipo: String = "2020-1-1",
    var marketCapitalization: Double,
    var exchange: String,
)

```

Kotlinstreaming.kt

```

package org.example

import kotlinx.serialization.decodeFromString
import kotlinx.serialization.json.Json
import org.apache.spark.SparkConf
import org.apache.spark.streaming.Durations
import org.apache.spark.streaming.api.java.JavaStreamingContext

fun main() {
    val streamIP = "localhost"
    val streamSocket = 1502
    val sparkConfig =
        SparkConf()
            .setMaster("local[*]")
            .setAppName("kotlin_streaming")
            .set("spark.executor.cores", "*")
    val streamingContext = JavaStreamingContext(sparkConfig,
Durations.seconds(3))

    val data = streamingContext.socketTextStream(streamIP, streamSocket)
    val dataJSON =
        data
            .map {
                Json { ignoreUnknownKeys = true }.decodeFromString<Data>(it)
            }.filter {
                it.exchange != "NEW YORK STOCK EXCHANGE, INC." && it.ipo != ""
&& it.phone != ""
            }.filter {
                2020 - it.ipo.split("-")[0].toInt() > 5
            }

    dataJSON.foreachRDD { dataRDD ->
        dataRDD.foreach {
            println("Nume:${it.name} \t Telefon:${it.phone} \t Piata de
capital:${it.marketCapitalization}")
        }
    }

    streamingContext.start()
    streamingContext.awaitTermination()
}

```

```
streamingContext.stop()
}
```

XML

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/maven-v4_0_0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>org.example</groupId>
    <artifactId>examen</artifactId>
    <version>1.0.0</version>
    <packaging>jar</packaging>

    <name>org.example examen</name>

    <properties>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <kotlin.version>1.4.0</kotlin.version>
        <kotlin.code.style>official</kotlin.code.style>
        <junit.version>4.12</junit.version>
        <serialization.version>1.0-M1-1.4.0-rc</serialization.version>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.jetbrains.kotlin</groupId>
            <artifactId>kotlin-maven-plugin</artifactId>
            <version>${kotlin.version}</version>
            <scope>provided</scope>
        </dependency>
        <!-- Apache RDD. -->
        <dependency>
            <groupId>org.apache.spark</groupId>
            <artifactId>spark-core_2.12</artifactId>
            <version>2.4.5</version>
        </dependency>
        <!-- Apache spark streaming. -->
        <dependency>
            <groupId>org.apache.spark</groupId>
            <artifactId>spark-streaming_2.12</artifactId>
```

```

        <version>2.4.5</version>
    </dependency>
    <!-- Kotlinx serialization JSON. -->
    <dependency>
        <groupId>org.jetbrains.kotlinx</groupId>
        <artifactId>kotlinx-serialization-runtime</artifactId>
        <version>${serialization.version}</version>
    </dependency>
    <dependency>
        <groupId>org.jetbrains.kotlin</groupId>
        <artifactId>kotlin-stdlib</artifactId>
        <version>${kotlin.version}</version>
    </dependency>
    <dependency>
        <groupId>org.jetbrains.kotlin</groupId>
        <artifactId>kotlin-test-junit</artifactId>
        <version>${kotlin.version}</version>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>junit</groupId>
        <artifactId>junit</artifactId>
        <version>${junit.version}</version>
        <scope>test</scope>
    </dependency>
</dependencies>

<build>
    <sourceDirectory>src/main/kotlin</sourceDirectory>
    <testSourceDirectory>src/test/kotlin</testSourceDirectory>

    <plugins>
        <plugin>
            <groupId>org.jetbrains.kotlin</groupId>
            <artifactId>kotlin-maven-plugin</artifactId>
            <version>${kotlin.version}</version>
            <executions>
                <execution>
                    <id>compile</id>
                    <phase>compile</phase>
                    <goals>
                        <goal>compile</goal>
                    </goals>
                </execution>
            </executions>
            <configuration>
                <compilerPlugins>
                    <plugin>kotlinx-serialization</plugin>
                </compilerPlugins>
            </configuration>
        </plugin>
    </plugins>
</build>

```

```

        </configuration>
        <dependencies>
            <dependency>
                <groupId>org.jetbrains.kotlin</groupId>
                <artifactId>kotlin-maven-serialization</artifactId>
                <version>1.8.22</version>
            </dependency>
        </dependencies>
    </plugin>
</plugins>
</build>
</project>

```

Server.py

```

import json
import socket
import time
from typing import List

import requests

class Api3rdParty:
    def __init__(self):
        self.__api_uri = "https://finnhub.io/api/v1/stock/"
        self.__token = "bs03ehfrh5rbvs9fgsag"
        self.api_url_stock_symbols =
'{}symbol?exchange=US&token={}'.format(self.__api_uri, self.__token)

    def getData(self) -> List[dict]:
        symbols_response = requests.get(self.api_url_stock_symbols)
        symbols_data = symbols_response.content
        return json.loads(symbols_data)

    def getSpecificData(self, json_obj: dict) -> dict:
        company_profile_url =
'{}profile2?symbol={}&token={}'.format(self.__api_uri, json_obj["symbol"],
self.__token)
        data = json.loads(requests.get(company_profile_url).content)
        return data if data.__ne__({}) else None

class ApiNetworkService:

```

```

__host: str = None
__port: int = None
__socket: socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
__client_socket: socket = None
__client_address: str = None

def __init__(self, host: str, port: int):
    self.__host = host
    self.__port = port

    self.__socket.bind((self.__host, self.__port))
    self.__socket.listen(1)
    self.__client_socket, self.__client_address = self.__socket.accept()

def sendJSON(self, json_data: str) -> None:
    self.__client_socket.sendall(json_data.encode())

def closeConnection(self) -> None:
    self.__client_socket.close()

if __name__ == "__main__":
    partyCon = Api3rdParty()
    serviceCon = ApiNetworkService('localhost', 1502)
    try:
        values = partyCon.getData()
        for item in values:
            profile_json_data = partyCon.getSpecificData(item)
            if profile_json_data is not None:
                print(json.dumps(profile_json_data))
                serviceCon.sendJSON(json.dumps(profile_json_data) + "\n")
                time.sleep(3)
    except Exception:
        serviceCon.closeConnection()
        exit(1)
    finally:
        serviceCon.closeConnection()

```